

DNS Subdomain Encoding

Necla MUTLU CURAL

June 14, 2025

1 Introduction

This project conducts a four-phase investigation into the "DNS Subdomain Encoding" covert channel, with a focus on its implementation, detection, and mitigation strategies. In Phase 1, a custom processor was developed to simulate randomized delays in Ethernet frames. Phase 2 involves the implementation of a covert communication mechanism using DNS Subdomain Encoding. In this method, arbitrary data are encoded into DNS query subdomains in the `insec` container, which are then transmitted to the `sec` container over the processor created in the Phase 1. Phase 3 addresses the challenge of detecting such covert channels. A machine learning-based detection system was designed and implemented using a Random Forest classifier. The model was trained on the [Malicious URLs Dataset](#), which includes labeled examples of benign and malicious domains. Feature engineering was applied to extract relevant characteristics from DNS subdomains, allowing the model to identify anomalous patterns indicative of covert activity. Finally, in Phase 4, a mitigation mechanism was introduced to limit the effectiveness of detected covert channels. A throttling mechanism was implemented to enforce rate limiting on IP addresses that generate excessive or suspicious DNS queries within a defined time window. This approach seeks to degrade the covert channel's bandwidth and reliability, thereby reducing its practical utility while minimizing disruption to legitimate network usage.

2 Phase 1

The processor was implemented using Python. Random delays using an exponential distribution with a mean delay of 5 microseconds ($5e-6$ s) were used in the experiment.

ICMP packets were captured and processed in an MITM setup, where they were subjected to the generated delay before being forwarded. The processor subscribed to `inpktsec` and `inpktinsec`, applied the random delay, and then published the modified frames to `outpktsec` or `outpktinsec`, depending on the direction.

The RTT values measured after applying the random delay are presented in Figure 1. Both the random delays and the RTT values were stored using Python's `pickle` module for later analysis.

3 Phase 2

3.1 DNS query sender implementation in `insec` container

This section describes the `dns-sender.py` script in the `insec` container to implement a covert communication channel using DNS queries. The script leverages the `Scapy` library to craft and send raw packets, enabling the transmission of hidden messages over legitimate DNS traffic.

3.1.1 Purpose

The script encodes a secret message, splits it into manageable chunks, and exfiltrates it covertly through the DNS subdomains.

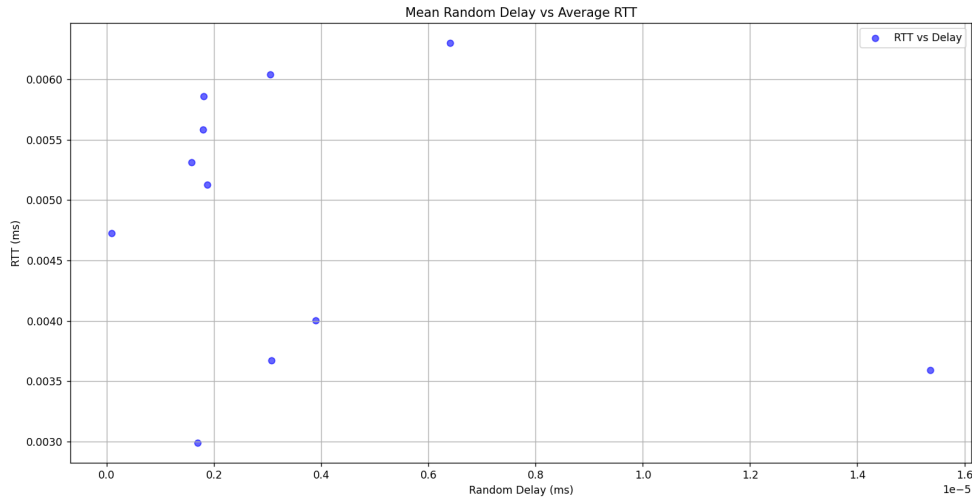


Figure 1: Phase 1: Delay vs. RTT

3.1.2 Encoding Function

The covert channel can use 3 different encodings depending on the "encoding" parameter. This function encodes each chunk of the message using the selected scheme:

```
def encode_message(msg, encoding_scheme):
    if encoding_scheme == 'base64':
        return base64.urlsafe_b64encode(msg.encode()).decode()
    elif encoding_scheme == 'hex':
        return msg.encode().hex()
    elif encoding_scheme == 'rot13':
        return codecs.encode(msg, 'rot_13')
```

3.1.3 Packet Construction and Sending

This function builds a DNS query packet:

```
def send_dns_query(domain, id):
    dns_query = Ether()/IP(dst='sec')/UDP(dport=53)/DNS(id=id, rd=1, qd=DNSQR(qname=domain))
    sendp(dns_query)
```

3.1.4 Main Execution Flow

The script expects 6 arguments to be provided:

1. Parses command-line arguments including:
 - **--message**: The secret message to transmit (required).
 - **--base-domain**: Domain suffix to append (e.g., `covert.local`).
 - **--chunk-size**: Number of bytes per DNS payload.
 - **--encoding**: Encoding scheme to use (`base64`, `hex`, `rot13`).
 - **--interval**: Time between packets in milliseconds.
 - **--count**: Maximum number of packets to send.
2. Splits the message into chunks of specified size.
3. Encodes each chunk using the selected encoding scheme.
4. Sends each chunk as a DNS query, one at a time, with a configurable delay between them.

3.2 DNS query receiver implementation

This section describes the `dns-query-receiver.py` script used in the `sec` container. The listener is a Python-based server that listens for incoming DNS queries on port 53 and sends back crafted DNS responses. This part is used to measure metrics such as average RTT, confidence interval margins, and covert channel capacity, all of which depend on the encoding scheme and fragment size used during message transmission.

3.3 Crafting and Sending DNS Responses

```
transaction_id = dns.id
dns_response = DNS(
    id=transaction_id,
    qr=1,
    aa=1,
    qd=dns.qd,
    an=DNSRR(rrname=qname, rdata="1.2.3.4", ttl=60)
)
```

The listener constructs a DNS response with the following characteristics:

- `id`: Matches the transaction ID of the query, which is crucial for performance benchmarking results.
- `qr=1`: Indicates that this is a response.
- `aa=1`: Marks the answer as authoritative.
- `qd`: Copies the question section from the request.
- `an`: Constructs a single answer record pointing to the dummy IP address 1.2.3.4.

Finally, the response is sent back to the origin address using:

```
sock.sendto(bytes(dns_response), address)
```

3.4 Experiments

In the experiments chunk sizes of 4, 8 and 16 are used with 3 different types of encodings `rot13`, `base64` and `hex`. The effect of the encoding scheme on the capacity and RTT for DNS queries is examined. The message is divided into equal chunks, the size of which is determined by the `chunk_size` parameter. Then chunks are encoded and sent to the `insec` container. The RTT time of packets between the processor and the `insec` container is measured. Capacity is calculated by dividing the total bytes transferred by RTT.

Table 1: Performance benchmark results

Encoding Scheme	Chunk Size	Average RTT	CI Margin	Average Capacity (bits)
base64	4	0.0089	0.00023	143.86
hex	4	0.0085	0.00023	123.86
rot13	4	0.0075	0.00030	209.83
base64	8	0.0065	0.00030	393.85
hex	8	0.0061	0.00026	336.13
rot13	8	0.0080	0.00039	370.11
base64	16	0.0078	0.00047	628.61
hex	16	0.0083	0.00041	458.55
rot13	16	0.0078	0.00044	690.38

ROT13 consistently yields the highest capacity, followed by `base64`, then `hex`. This might be due to the fact that ROT13 is a simple letter substitution and keeps character size the same. `Base64` expands

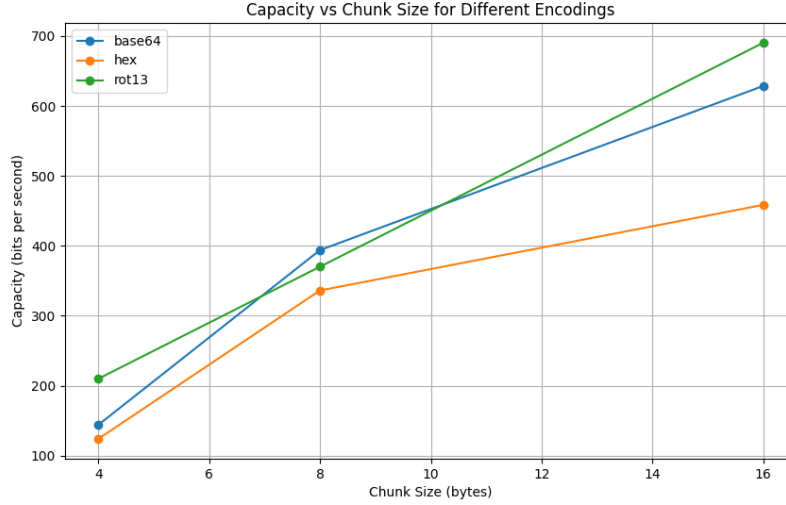


Figure 2: Capacity vs Chunk Size for Different Encodings

the data (3 bytes become 4 characters), which slightly reduces efficiency. Hex expands the data the most (1 byte becomes 2 characters), so it performs the worst in capacity.

At very small chunk sizes, more DNS queries are needed to send the full message, which increased overhead. At very large chunk sizes, each individual DNS packet is heavier and might trigger longer processing times or fragmentation, slightly raising RTT. At a medium chunk size (8 bytes), it hits a spot where fewer requests without overloading each one.

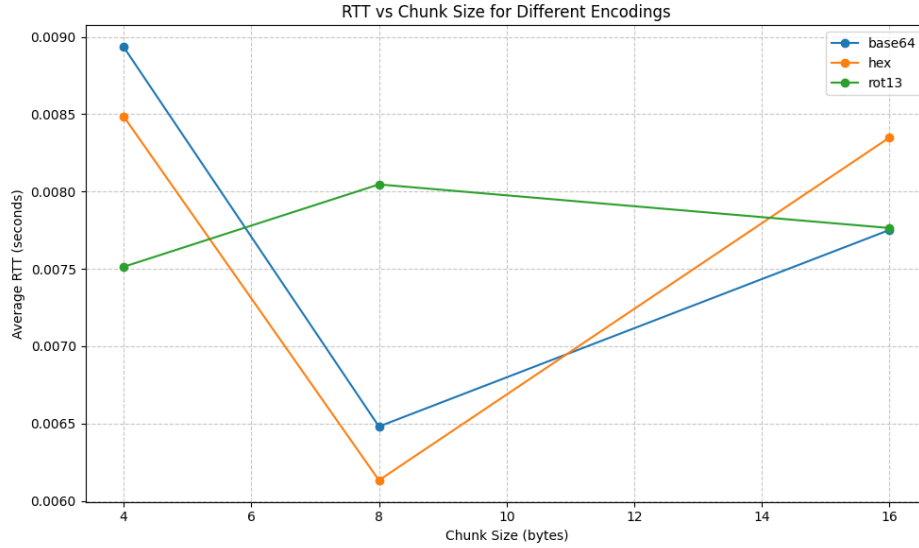


Figure 3: RTT vs Chunk Size for Different Encodings

4 Phase 3

A machine learning-based detection system was designed and implemented using a Random Forest classifier. The model was trained on the [Malicious URLs Dataset](#), which includes labeled examples of benign and malicious domains. Feature engineering was applied to extract relevant characteristics from DNS subdomains.

4.1 Feature engineering

To facilitate the classification of URLs, a set of handcrafted features was extracted. These features are defined as follows:

- **url_len**: The total number of characters in the URL string.
- **uppercase_ratio**: The ratio of uppercase alphabetic characters to the total length of the URL.
- **digit_ratio**: The proportion of numeric digits present in the URL. A high digit ratio can be indicative of algorithmically generated domains or encoded payloads.
- **total_label_ratio**: The normalized length of the entire URL with respect to a predefined maximum length constant (**MAX_TOTAL_LENGTH**).
- **per_label_ratio**: The maximum label length within the URL (i.e., the length of the longest subdomain or domain label), normalized by a predefined constant (**MAX_LABEL_LENGTH**).
- **entropy**: The maximum Shannon entropy value computed across the entire URL string and each of its individual labels. Entropy serves as a statistical measure of randomness, with higher values typically suggesting domain name obfuscation or the use of encoded strings.

4.2 Random Forest Classifier

Random Forest Classifier were used as the detector from scikit-learn library.

4.3 Experiments

To evaluate the performance of the covert channel detector, a synthetic DNS domain dataset was generated using different encodings. The script simulated encoded and non-encoded DNS subdomains and labeled them for supervised learning evaluation. The methodology is outlined below:

- The subdomain portion of DNS queries was randomly manipulated using three base64, hex or rot13 encoding schemes.
- Each domain consisted of up to four subdomain segments. Each segment could be a predefined keyword (e.g., **www**, **cdn**, **login**), a randomly generated alphanumeric string, or encoded string (with 80% probability). These subdomain segments were concatenated with randomly selected popular domain names (e.g., **google.com**, **github.com**) to form realistic-looking DNS queries. The ones including encoded strings are labeled as **malign** and others are labeled as **benign**.

The evaluation was conducted in four distinct phases to assess the detector’s performance when different encodings are used. In the first experiment, a balanced dataset consisting of 10,000 domain names was constructed, where each sample had an equal probability of being unencoded, Base64-encoded, Hex-encoded, or ROT13-encoded. In other experiments, it was aimed to evaluate the detector’s sensitivity and specificity to individual encoding schemes, three additional datasets were generated, each containing 10,000 samples exclusively using a single encoding method (Base64, Hex, or ROT13). These datasets enabled a focused analysis of the classifier’s performance on each encoding type in isolation.

Table 2: Evaluation Metrics by Encoding Scheme

Encoding Scheme	TP	TN	FP	FN	Precision	Recall	F1-Score
Mixed	3024	1602	3402	1972	0.47	0.61	0.53
Base64	4483	1641	3339	537	0.57	0.89	0.70
Hex	3089	1663	3354	1894	0.48	0.62	0.54
ROT13	4073	1660	3391	876	0.55	0.82	0.66

4.4 Discussion

The overall performance of the URL-based detector model was observed to be suboptimal. One plausible explanation for this shortcoming is the inherent mismatch between the malign URL dataset used for training and the actual characteristics of DNS-based covert channels, particularly those constructed through subdomain encoding. The dataset predominantly contains malicious URLs related to phishing, spam, or malware distribution, which may differ significantly in structure and behavior from synthetically generated DNS queries employed in covert communication.

Moreover, it was noted that the model demonstrated comparatively better performance when detecting DNS subdomains encoded using the Base64 scheme. This suggests that the lexical and statistical patterns introduced by Base64 encoding may be more distinguishable by the current feature set and learning algorithm.

5 Phase 4

5.1 Mitigation strategy: Source-Based Throttling

To mitigate the impact of the covert DNS subdomain channel, a source-based throttling mechanism was implemented. This approach aims to restrict the bandwidth available to potential attackers by rate-limiting the number of suspicious DNS queries allowed from a single source IP address within a defined time window.

For each DNS query, the detection system described in Phase 3 is invoked to classify the domain name as benign or potentially malign.

If a DNS query is flagged as malign, the system records the source IP and evaluates whether the request should be dropped based on two configurable parameters:

- **Throttle Limit (L):** The maximum number of suspicious requests allowed from a single source IP.
- **Time Window (T):** The duration (in seconds) over which the L requests are tracked.

A request is dropped (i.e., not forwarded) if the source IP has already issued L or more encoded queries in the past T seconds. Otherwise, it is allowed and counted toward the total allowed encoded requests from that IP.

Additionally, the system tracks two key metrics:

- **Total Encoded Queries Received:** Number of encoded queries detected per source.
- **Total Encoded Queries Allowed:** Number of encoded queries permitted under the throttling threshold.

Using these metrics, the effective **covert channel capacity** is calculated periodically as:

$$\text{Capacity (bps)} = \frac{\sum_i (\text{Allowed}_i \times \text{chunk_size}_i \times 8)}{\text{Elapsed Time (s)}}$$

where chunk_size_i denotes the number of characters used to encode data per DNS query for source i . This provides a real-time estimate of the throughput achieved by the covert channel after throttling is applied.

5.2 Performance benchmark result

The experimental results demonstrate that throttling is an effective mitigation strategy for limiting covert channel capacity over DNS traffic. When a small throttle limit (e.g., 5 requests per 10 seconds) is enforced, the bandwidth of the channel is significantly reduced, dropping to approximately 32 bits/sec on average from an unrestricted rate of around 154 bits/sec. As the throttle limit increases, the covert channel gradually regains its original capacity, indicating that the rate limit directly governs the throughput of encoded traffic.

On the other hand, varying the time window while keeping the throttle limit fixed at 5 reveals a different behavior. A very short time window (e.g., 5 seconds) allows more frequent resets of the

Table 3: Effect of Throttle Limit on Covert Channel Capacity

Throttle Limit	Mean	Std Dev	CI Lower	CI Upper
0	154.56	1.53	153.46	155.66
5	31.69	0.71	31.18	32.20
10	63.75	0.45	63.43	64.07
15	95.99	0.90	95.35	96.64
20	114.73	0.87	114.11	115.35
25	153.25	1.18	152.41	154.09

Table 4: Effect of Time Window on Covert Channel Capacity

Time Window	Mean	Std Dev	CI Lower	CI Upper
5	63.44	0.75	62.90	63.98
10	31.69	0.71	31.18	32.20
15	31.43	0.47	31.09	31.77
20	31.43	0.50	31.07	31.79
25	31.24	0.33	31.01	31.47

request counter, enabling slightly more data to be transmitted. However, as the window increases beyond 10 seconds, the time window ceases to be the primary bottleneck. In these cases, the throttle limit becomes the dominant constraint, and the covert channel capacity stabilizes around the same value.

These findings suggest that the throttle limit parameter is more influential than the time window in shaping covert channel performance. For robust mitigation, a combined configuration of a moderate time window (e.g., ≥ 10 seconds) and a relatively strict throttle limit (e.g., ≤ 10 requests) effectively reduces the risk of covert exfiltration without significantly disrupting legitimate DNS activity.

6 Conclusion

This project investigated the detection and mitigation of covert communication channels exploiting DNS query subdomains. A machine learning-based detector was developed and trained to identify suspicious domain patterns, including various encoding schemes such as Base64, hexadecimal, and ROT13. The suboptimal performance of the detector can be attributed to the nature of its training dataset, which primarily consisted of malicious URLs rather than domain-specific DNS query patterns. As a result, the learned features may not effectively generalize to the structure and characteristics of encoded DNS subdomains. Performance could be improved by constructing a training dataset that specifically reflects the distribution and encoding schemes observed in DNS-based covert channels.

In the final phase, a throttling-based mitigation strategy was implemented to reduce the bandwidth of potential covert channels. Experiments were conducted to measure how different throttling parameters affect the covert channel capacity. Results showed that limiting the number of allowed queries per time window significantly constrains the channel’s throughput. In particular, a lower throttle limit proved to be highly effective in reducing capacity, while the impact of the time window was less pronounced once it reached a moderate duration.